Guia de integração

Campeonatos + placar eletrônico

DESENVOLVIMENTO DE SISTEMAS | 2º ANO | 3º TRIMESTRE

Objetivo da atividade

Construir dois sistemas independentes que se comunicam por API REST. O gerenciamento de campeonatos cria uma partida e acompanha o resultado; o placar eletrônico gerencia pontuação, tempo, faltas e etapas.

Arquitetura definida

Componente	Responsabilidade
Campeonatos	Cadastra partida, guarda o ID e consulta resultado
Placar (Flask)	Disponibiliza POST, GET e PATCH
Docker na escola	Executa a API na rede local
Supabase	PostgreSQL, com bancos separados para os grupos
GitHub	Versionamento do código e documentação

A máquina com Docker deve ter IP estável/reserva DHCP e permitir conexões entre computadores. É necessária internet para acessar o Supabase; as credenciais devem ficar somente no backend.

Repositório-base

github.com/rvarjao/placar-eletronico-api

O projeto já contém a estrutura Flask, arquivos de rotas, modelos, serviços e repositórios, Docker e documentação. As rotas devolvem HTTP 501 (sem implementação), para os estudantes desenvolverem a lógica.

Contrato da API

Campos e valores padronizados

Estrutura de dados

Campo	Tipo / regra
id	inteiro gerado no servidor
nome_equipe_a / nome_equipe_b	textos obrigatórios
modalidade	texto validado: futebol, futsal, basquete, volei, handebol, outro
status	nao_iniciado, em_andamento, pausado, finalizado
tempo	inteiro em segundos, maior ou igual a zero
etapa	texto: 1º tempo, 3º quarto, 4º set, intervalo etc.
pontos_equipe_a / pontos_equipe_b	inteiros não negativos
faltas_equipe_a / faltas_equipe_b	inteiros não negativos
detalhes	objeto JSON com informações específicas do esporte

Rotas obrigatórias

Método e caminho	Comportamento
POST /api/placares	Cria o placar; retorna 201 e o ID
GET /api/placares/{id}	Retorna estado atual; 200 ou 404
PATCH /api/placares/{id}	Atualiza campos enviados; retorna 200

Sugestão: usar VARCHAR com valores validados na aplicação para modalidade e status; deixar etapa como texto livre. O grupo do placar responde pelos controles da partida.

Tratamento de erros

201 criado, 200 sucesso, 400/422 entrada inválida, 401/403 sem autorização, 404 não encontrado, 500 erro inesperado, 503 indisponibilidade do serviço. Respostas de erro devem ser JSON.

Exemplos de requisição

Criar, consultar e atualizar

POST /api/placares - criação

```
{
  "nome_equipe_a": "Brasil",
  "nome_equipe_b": "Argentina",
  "modalidade": "futebol",
  "etapa": "1º tempo"
}
```

Resposta HTTP 201: a API devolve o objeto com id (ex.: 15), status nao_iniciado, tempo 0, pontuações 0, faltas 0 e detalhes {}. A aplicação de campeonatos guarda esse ID em sua própria base.

GET /api/placares/15 - consulta

```
{
  "id": 15,
  "nome_equipe_a": "Brasil",
  "nome_equipe_b": "Argentina",
  "modalidade": "futebol",
  "status": "em_andamento",
  "tempo": 2520,
  "etapa": "2º tempo",
  "pontos_equipe_a": 2,
  "pontos_equipe_b": 1,
  "faltas_equipe_a": 3,
  "faltas_equipe_b": 5,
  "detalhes": {}
}
```

PATCH /api/placares/15 - atualização

```
{ "pontos_equipe_a": 3, "tempo": 2700, "etapa": "2º tempo" }
```

A rota PATCH recebe só os campos alterados, valida as alterações e responde com o objeto completo atualizado. O ID não pode ser modificado.

Comunicação e interfaces

Integração entre os dois grupos

Fluxo didático por polling

1. Campeonatos cadastra a partida e faz POST na API do placar.
2. O placar retorna o ID; campeonatos salva placar_id localmente.
3. Campeonatos executa GET /api/placares/{id} a cada 5 segundos.
4. Exibe os dados retornados e interrompe as consultas quando status = finalizado.
5. Em falha de rede, mostra aviso e tenta novamente; não cria outro placar automaticamente.

Interface genérica + layouts por modalidade

Modalidade	Elementos principais
Futebol e futsal	Gols, 1º/2º tempo, cronômetro crescente e faltas
Basquete	Pontos, quartos, tempo regressivo e faltas
Vôlei	Pontos do set atual, sets vencidos e etapa

No vôlei, pontos_equipe_a/b representam pontos do set em andamento. Os sets vencidos podem ficar em detalhes: {"sets_equipe_a":2,"sets_equipe_b":1}. Não é necessário criar aplicações distintas por modalidade: reutilizar componentes e adaptar a interface.

Etapas e critérios de entrega

Fazer fork, modelar o Supabase, implementar rotas e testes, desenvolver interface genérica, integrar por HTTP e especializar modalidades. Demonstrar criação, atualização, consulta, encerramento e tratamento de erros. O grupo de campeonatos não deve acessar diretamente o banco do placar.